GenResearch-Servidor

Memória, armazenamento, sincronização e desempenho em um serviço RAG

Sistemas Operacionais — AV2 — 2026.2 · Leticia Cavalcanti (individual)

Pergunta: *como memória, armazenamento, caches, filas e sincronização afetam desempenho, estabilidade e correção de uma aplicação de IA generativa/RAG?*

1. Problema e recorte

- RAG mobiliza **arquivos grandes**, **memória virtual**, **cache de páginas**, **filas**, **caches** e **logs** concorrentes
- Foco: **o que o SO faz** — não a qualidade da resposta
- Recorte: serviço próprio, **vetores e geração simulados**, corpus **sintético** (sem chave de API, sem pesos, reprodutível)
- Configurações **C1–C6** + Parte D (sincronização) + Parte E (memória virtual), 3 repetições cada

2. Ambiente

SO	Debian 13 em **contêiner** (Docker Desktop, WSL2)
Kernel	6.6.87.2-microsoft-standard-WSL2
CPU / RAM / swap	i5-10210U, 4 CPUs lógicas · 5,8 GiB · 1 GiB
GPU	nenhuma
Pilha	Python 3.12 · numpy 2.1.3 · SQLite

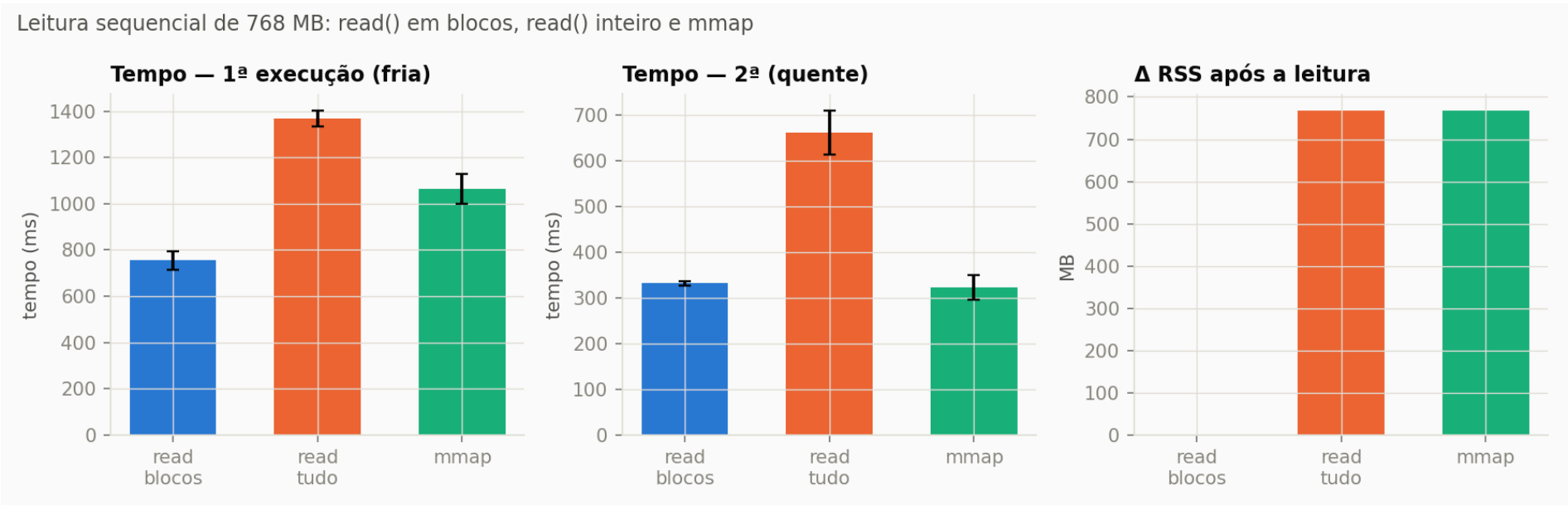
Comandos: `uname`, `lscpu`, `free`, `/proc/meminfo`, `lsblk`, `df`, `vmstat`, `iostat` → `results/env/`

3. Arquitetura

```
2 produtores → BoundedBuffer (empty/full/mutex) → N threads trabalhadoras
  cache (LRU/FIFO + mutex) → embedding → varredura do índice (.npy: read | mmap)
  → SQLite (textos) → contexto limitado → geração simulada → log (mutex)
```

- Threads: carga **mista** (sleep/numpy/hashlib liberam o GIL)
- Recursos compartilhados: **fila, cache, log, conexões SQLite**

4. C1 × C2: `read()` × `mmap` (768 MB)



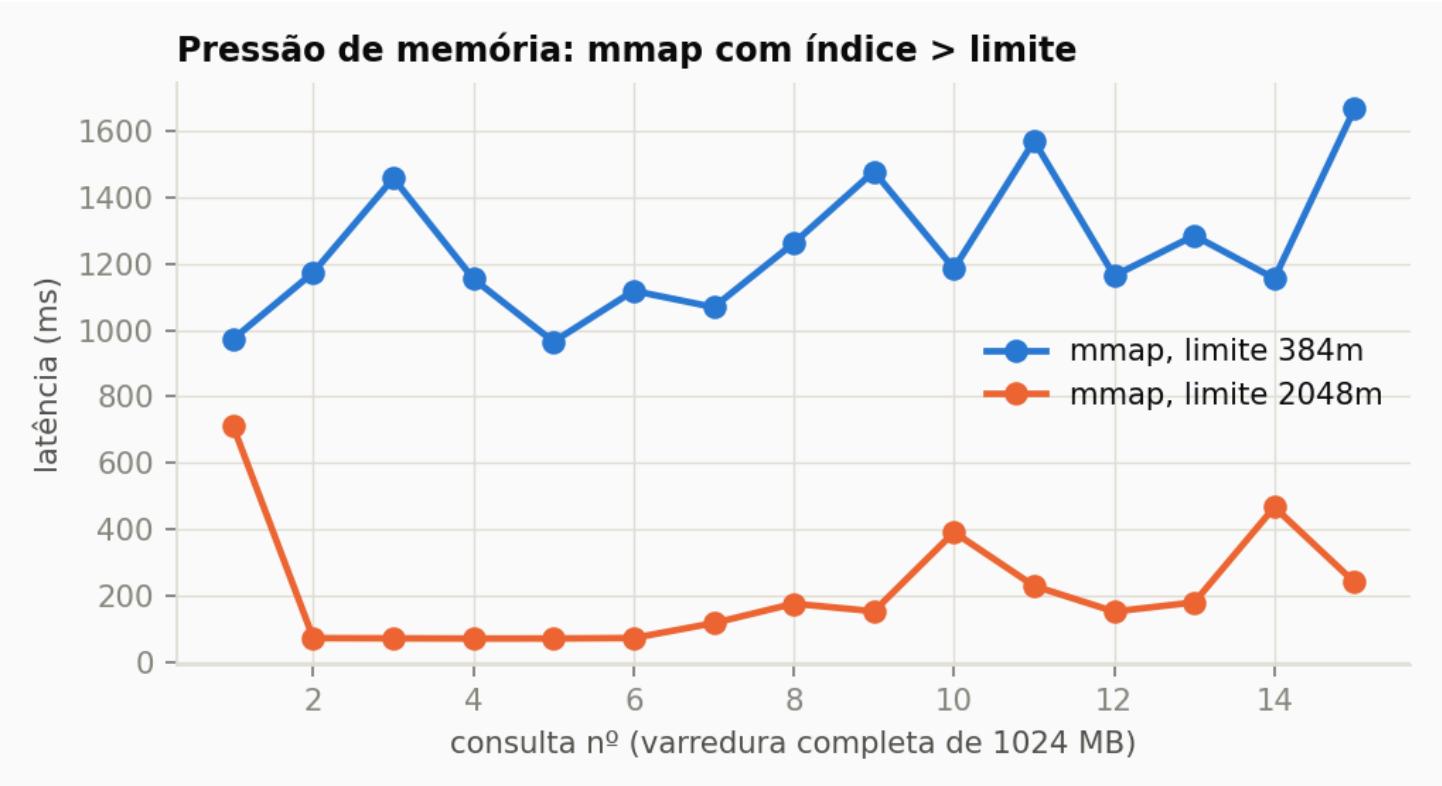
- **2ª execução 2–3× mais rápida**, 0 MB lidos do disco → **cache de páginas**
- `read()` do arquivo inteiro: **+768 MB de RSS** e 2× mais lento que ler em blocos
- `mmap`: VSZ sobe na abertura, RSS só ao tocar; 6.146 faltas maiores (~125 KiB cada, *readahead*)

5. mmap em acesso aleatório frio

- 3.000 acessos de 4 KiB (≈ 12 MB úteis)
- `pread`: **23 MB** lidos do disco · `mmap`: **287 MB** (12× de amplificação) e 2,2× mais lento
- Quente: diferença pequena (12,8 × 17,6 ms)

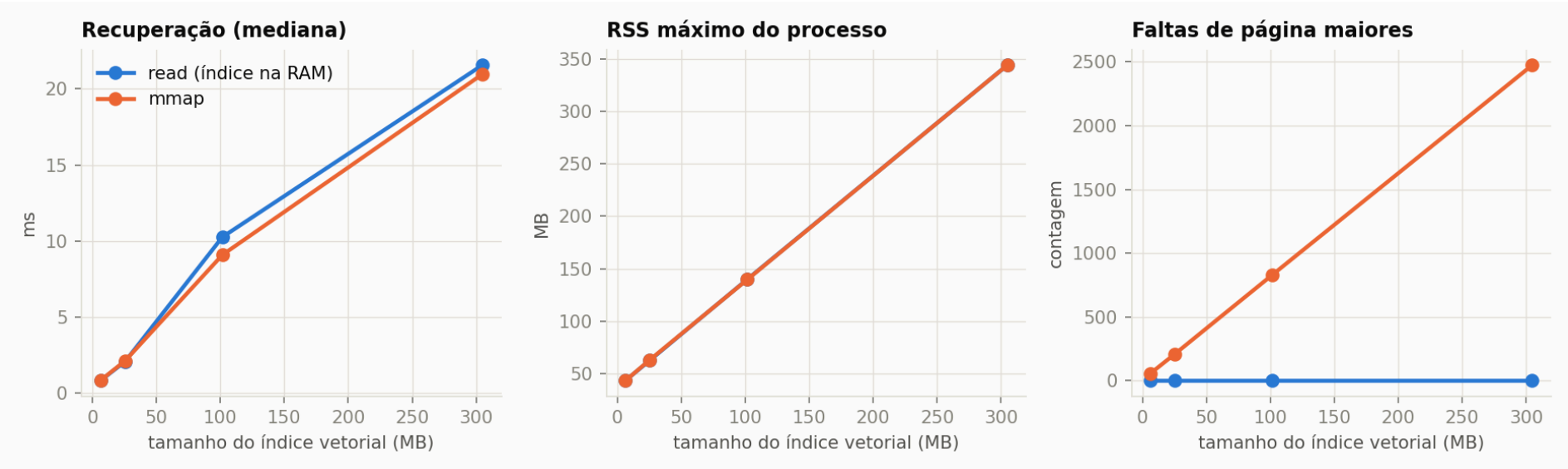
→ *mmap não é vantajoso para acesso esparso e frio*

6. C5: índice maior que a memória



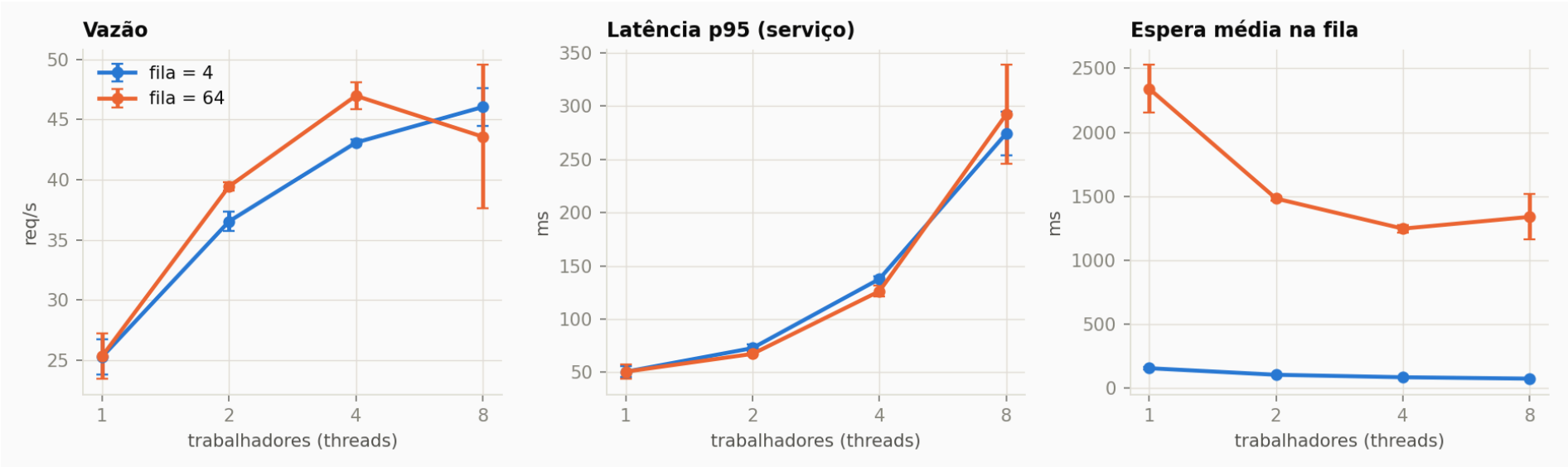
- Índice 1 GB, limite **384 MB**: `read` → **OOM kill (137)** · `mmap` → **sobrevive**
- Mas com **thrashing**: ~125 mil faltas maiores, **15,3 GB lidos**, consulta **13–17× mais lenta**
- Swap **não** foi exercitado (limite sem swap)

7. C5: crescimento do corpus e do contexto



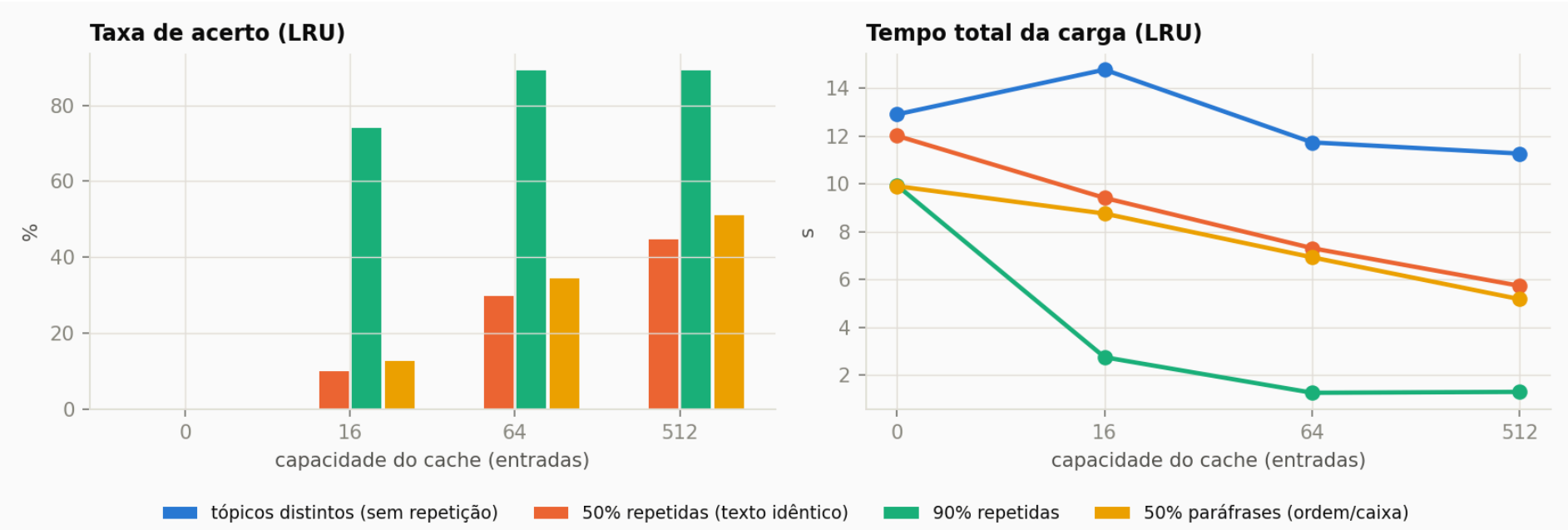
- Recuperação **linear** no índice (≈ 70 µs/MB); RSS ≈ índice + ~40 MB
- `read` e `mmap` iguais em regime estacionário; muda **quando** o custo é pago
- Contexto 500 → 8.000 B: geração 20 → ~110 ms

8. C3: concorrência e fila



- 4 workers: **1,7–1,85×** a vazão · 8 workers: **sem ganho**, p95 **~5× pior**
- Fila 64: espera de **1,2–2,3 s** (fila 4: ~0,1 s) → fila grande = latência, não vazão
- 0 perdas, 0 duplicatas, 0 violações FIFO

9. C4: cache compartilhado



- 90% de repetição: **89,2% de acerto**, tempo **9,9 s → 1,25 s** (≈ 8×)
- Consultas distintas: **0%** de acerto · LRU ≈ FIFO (repetição uniforme)

10. Sincronização e falha controlada

Demonstração	Sem proteção	Com proteção
Cache: atualizações perdidas	**86%**	0 (mutex / lock por chave)
Cache: computações duplicadas	22	0
Log: linhas corrompidas	**76%**	0
Deadlock A→B / B→A (subprocesso)	travou → **kill** após 3 s	ordenação de locks: 0,3 s · timeout+recuo: 0,6 s

Threads travadas: `futex_wait_queue` · lock por chave: −18% de tempo vs. mutex global

11. C6: falhas controladas

- Índice ausente: 40 erros tratados, 80 retentativas, processo estável
- Índice volta em 0,6 s: **120/120** concluídas com retentativas
- Fila cheia: **rejeita** (112 de 120) × **bloqueia** (120 ok, 4 s)
- Serviço externo 30% falho: **34% de erro** → **0,8%** com 3 retentativas (+88% de tempo)
- Limite de memória: OOM kill do `read`

12. Relação com IA generativa

- Carregar *embeddings/checkpoints*: `read()` = memória anônima; `mmap` = paginação sob demanda (e *thrashing* se não cabe)
- 1ª carga fria ≫ próximas (cache de páginas)
- Força bruta O(N) → índices aproximados; contexto = custo **e** qualidade
- Filas limitadas = *backpressure*; caches de resposta só pagam com repetição

13. Limitações

- Contêiner sobre WSL2: "frio" = frio no cache da VM · 2 núcleos físicos, com ruído
- Geração e embeddings **simulados** → não generaliza para GPUs, modelos grandes ou ambiente distribuído
- Corrida com **injeção de falha** (janelas ampliadas) · swap e starvation não testados
- Causa total da saturação com >2 workers não isolada (OpenBLAS explica ~10–15%)

Obrigada

Repositório, relatório, dados, logs e instruções em `README.md` · `python scripts/run_all.py`